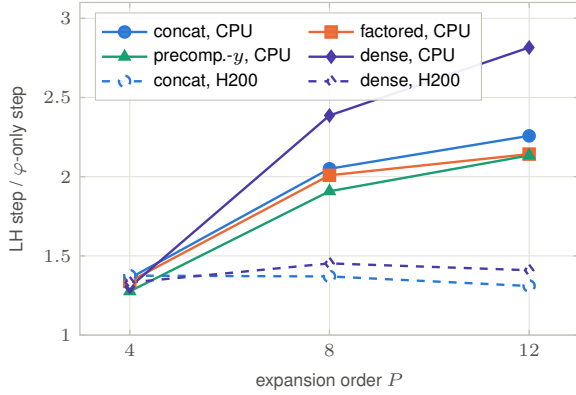
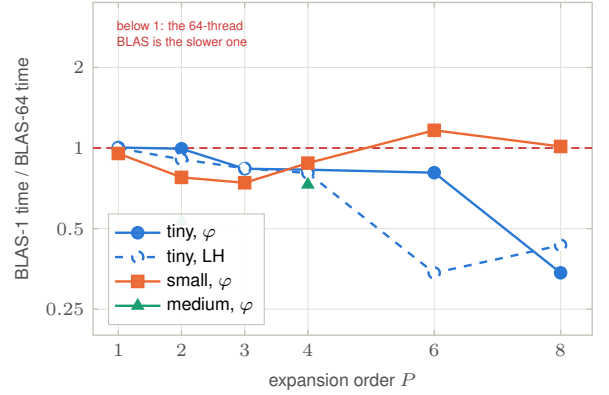


Fig. 6 — What the speedups cost: the χ channel, multi-thread BLAS, construction, and accuracy

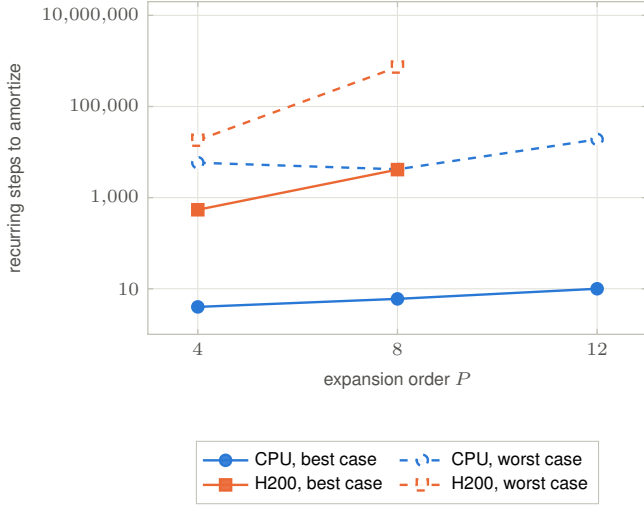
(a) Lamb–Helmholtz cost multiplier



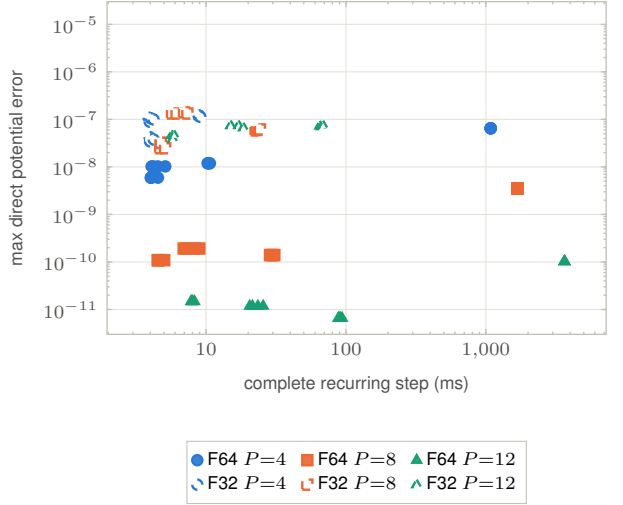
(b) 64-thread BLAS vs single-thread BLAS



(c) dense construction break-even



(d) accuracy vs cost (H200)



Machines / regimes: (a,c,d) task 024 — CPU m12-2-17 EPYC 7763 (BLAS 1 and 64, julia_threads=1), GPU m13h-1-1 H200. (b) task 019b, m12-1-7 EPYC 7763, whole-slab concat host stage, Float64. **Data:** operator_ab_benchmark/summary/{case_rankings,construction_amortization}.csv, operator_ab_benchmark/raw/cuda_*.csv, smallp_fallback_layout/m12-1-7/crossover_stage_blas{1,64}.csv (tables fig06a_*...fig06d_*). **Reads as:** carrying the Lamb–Helmholtz χ channel costs $1.3 \times$ at $P = 4$ rising to $2.1\text{--}2.8 \times$ at $P = 12$, and it is the axis that most often flips the strategy choice in Fig. 5. Handling the operators a 64-thread BLAS *loses* up to $3\text{--}4 \times$ at small route counts, which is why the standing rule is to pin BLAS to one thread inside the operators rather than to add a fallback path. The dense strategy's construction amortizes in $4\text{--}220$ steps across the measured large- n CPU cases ($4\text{--}35$ for uniform Float64, $6\text{--}109$ for clustered Float64, and $52\text{--}220$ for uniform Float32), but needs $5 \times 10^2\text{--}7 \times 10^5$ steps on the H200 — the reason precomputed- y is the recommended GPU default despite dense winning many individual steps. **Scope:** (a,c,d) use the complete recurring $\mathbb{fmm}$! step, far field *and* nearfield (the latter inside L2B); panel (b) is the far-field M2L host stage alone. Because the constant- P stencil promotes more pairs to the far field as P grows, the nearfield share of (a,c,d) shrinks with P — at $n = 20000$ the direct-pair count falls from 227 632 at $P = 4$ to 39 496 at $P = 12$ — so the two costs trade against each other along the P axis. **Caveats:** panel (d) is not an accuracy-vs-tolerance study. Every 024 case ran a single fixed ConstantPAnalyticStencil tolerance, so the only accuracy axis available is Float32 vs Float64 at each P ; the spread along x within one series is the four strategies and the three problem sizes, not an accuracy knob. No stencil-tolerance sweep exists anywhere in the project's data — tolerance-vs-cost remains unmeasured and is carried to the 019a review as a limitation. In (c), $P = 12$ has no GPU point because dense is not a steady-state winner against precomputed- y there; the CPU worst case at each P is dominated by the $n = 150$ cases (up to 18 930 steps). Panel (b) gaps at $P \in \{1, 3, 6, 8\}$ for the medium configuration are unrun points.